

Execução Consciente em Python - Como o Interpretador Executa Código e Como Ler Erros com Precisão Técnica

Introdução

A execução de um programa em Python é um processo técnico, sequencial e determinístico conduzido pelo interpretador da linguagem. Não há inferência automática, antecipação de intenções ou correção implícita de falhas. O interpretador executa exatamente o que está formalmente escrito no código.

Antes de avançar para estruturas condicionais, operadores lógicos, estruturas de repetição ou modularização, é fundamental compreender como ocorre a leitura e execução de um arquivo Python. Essa compreensão estabelece a base conceitual necessária para analisar comportamentos inesperados e interpretar corretamente mensagens de erro.

A execução de um programa ocorre linha por linha, respeitando a ordem em que as instruções foram escritas. Caso uma falha seja encontrada, o interpretador interrompe imediatamente o processamento. Nenhuma instrução posterior é executada após a ocorrência do erro.

Esse comportamento não é um detalhe secundário da linguagem; é uma característica estrutural do seu funcionamento. Em sistemas reais, uma interrupção de execução pode impedir a conclusão de processos como cálculos financeiros, atualizações de banco de dados ou geração de relatórios. Portanto, compreender o mecanismo de execução não é apenas um conteúdo introdutório, mas um fundamento técnico.

Esta seção estabelece:

- O modelo formal de execução sequencial do Python
- A lógica de interrupção em caso de erro
- A distinção entre erro estrutural (erro de sintaxe) e erro em tempo de execução
- A leitura técnica das mensagens exibidas no terminal

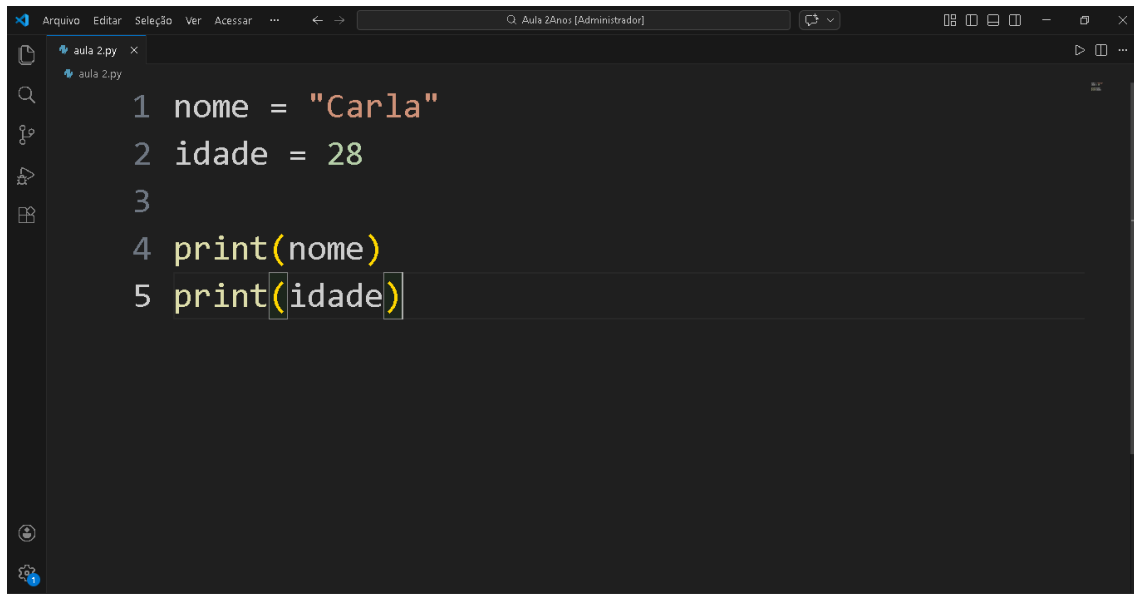
O objetivo é construir um entendimento preciso do comportamento do interpretador, desenvolvendo capacidade de análise estruturada diante de falhas no código.

Execução Sequencial do Código em Python

A execução de um programa em Python ocorre de forma sequencial. Isso significa que o interpretador lê o arquivo de cima para baixo, executando cada instrução na ordem exata em que foi escrita.

Não há salto automático entre linhas, não há reorganização interna do código e não há antecipação de comandos futuros. A ordem das instruções define o comportamento do programa.

Considere o exemplo a seguir:



```
1 nome = "Carla"
2 idade = 28
3
4 print(nome)
5 print(idade)
```

Análise da Execução

1. O interpretador lê a primeira linha e cria um objeto do tipo string contendo o valor "Carla", associando-o à variável nome.
2. Em seguida, lê a segunda linha, cria um objeto inteiro com valor 28 e o associa à variável idade.
3. A terceira linha executa a função print, exibindo o valor armazenado em nome.
4. A quarta linha executa outra chamada de print, exibindo o valor armazenado em idade.

Cada linha depende do que foi definido anteriormente. Se a variável nome não tivesse sido criada antes da chamada de print(nome), ocorreria um erro.

A execução sequencial implica três consequências importantes:

- Variáveis precisam ser definidas antes de serem utilizadas.
- Operações dependem do estado atual das variáveis no momento da execução.
- A ordem das instruções altera diretamente o resultado final do programa.

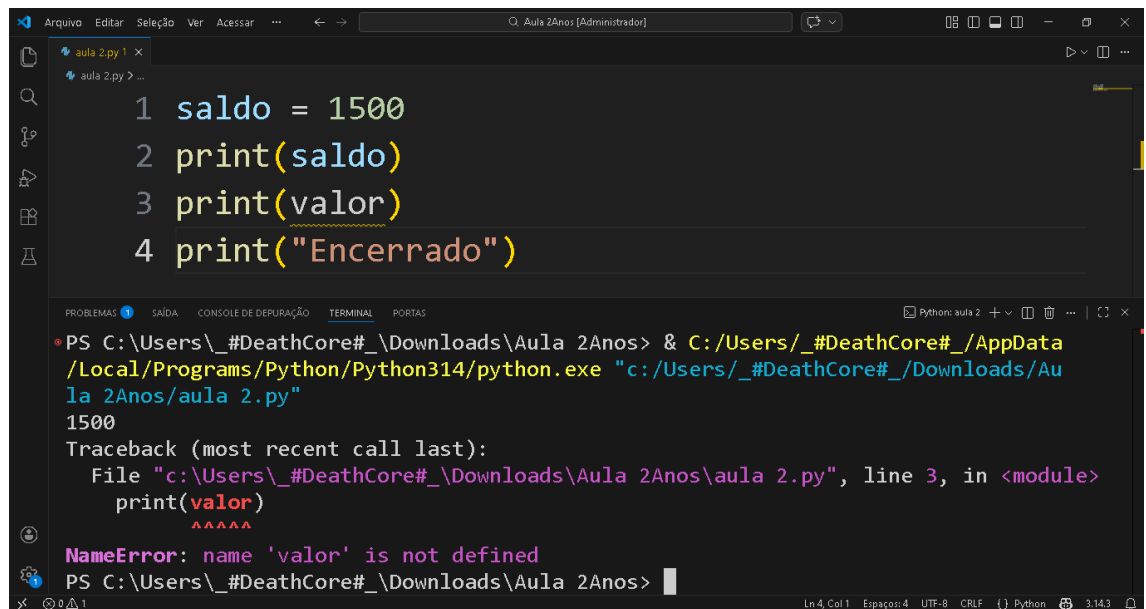
Esse modelo de leitura linear deve ser compreendido de forma clara, pois toda a estrutura da linguagem Python — inclusive erros — depende desse comportamento.

Interrupção da Execução em Caso de Erro

Durante a execução sequencial, o interpretador Python mantém um fluxo contínuo de leitura e processamento das instruções. No entanto, se uma falha ocorrer em determinada linha, a execução é interrompida imediatamente naquele ponto.

Isso significa que nenhuma instrução posterior será executada após a ocorrência do erro.

Considere o exemplo:



```
Arquivo  Editar  Seleção  Ver  Acessar  ...  Aula 2Anos [Administrador]
aula 2.py 1 x
aula 2.py > ...
1 saldo = 1500
2 print(saldo)
3 print(valor)
4 print("Encerrado")

PROBLEMAS  SAÍDA  CONSOLE DE DEPURAÇÃO  TERMINAL  PORTAS
Python: aula 2 + ~  Python: aula 2 + ~  Python: aula 2 + ~  Python: aula 2 + ~
* PS C:\Users\_#DeathCore#\Downloads\Aula 2Anos> & C:/Users/_#DeathCore#/AppData
/Local/Programs/Python/Python314/python.exe "c:/Users/_#DeathCore#/Downloads/Au
la 2Anos/aula 2.py"
1500
Traceback (most recent call last):
  File "c:\Users\_#DeathCore#\Downloads\Aula 2Anos\aula 2.py", line 3, in <module>
    print(valor)
          ^^^^^
NameError: name 'valor' is not defined
PS C:\Users\_#DeathCore#\Downloads\Aula 2Anos>
```

Análise do Fluxo de Execução

1. A variável saldo é criada com valor 1500.
2. O valor de saldo é exibido corretamente.
3. O interpretador tenta executar print(valor).
4. Como a variável valor não foi definida anteriormente, ocorre um erro do tipo:

NameError (Erro de Nome) — variável utilizada sem ter sido declarada.

A partir desse ponto:

- A linha que contém "Encerrado" não será executada.
- O programa é finalizado imediatamente após o erro.

Representação do Fluxo

Linha 1 → Executa

Linha 2 → Executa

Linha 3 → Erro (NameError)

Linha 4 → Não executa

Essa interrupção não é opcional nem configurável nesse estágio básico da linguagem. Ela faz parte do comportamento padrão do interpretador.

Esse mecanismo demonstra que:

- A execução depende da validade de cada instrução.
- Um único erro compromete todo o restante do processamento.
- A análise do ponto exato da falha é essencial para correção adequada.

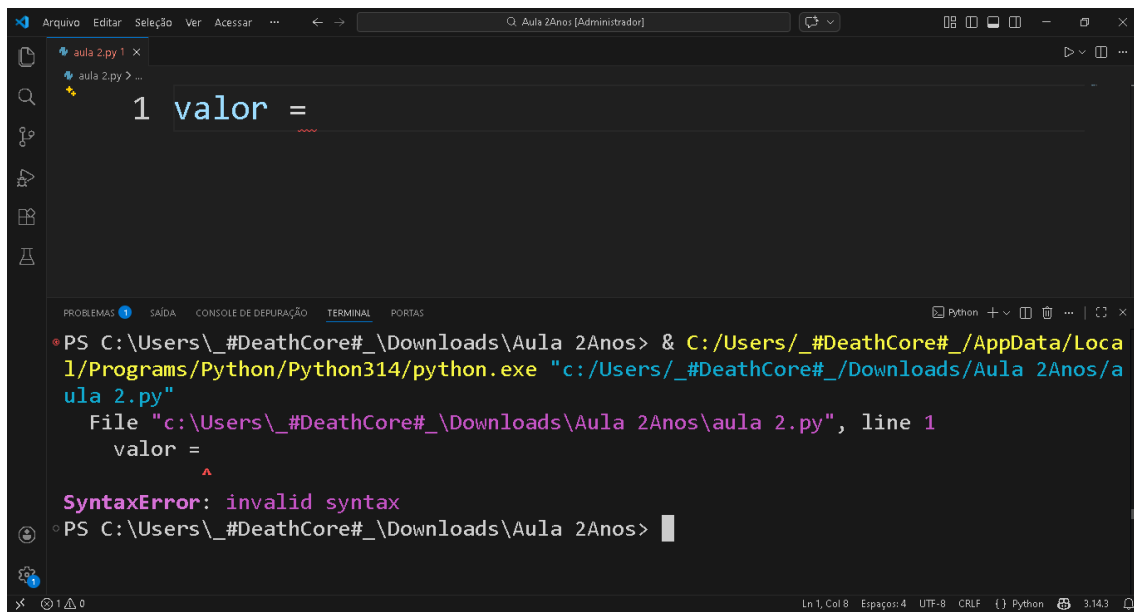
A partir dessa compreensão, torna-se possível diferenciar dois grandes grupos de erros: erros estruturais (de sintaxe) e erros que ocorrem durante a execução.

Erro de Sintaxe e Erro em Tempo de Execução

Nem todo erro ocorre da mesma maneira. Em Python, é possível distinguir dois grupos principais: erros estruturais (de sintaxe) e erros que ocorrem durante a execução do programa.

Erro de Sintaxe (SyntaxError — Erro de Sintaxe)

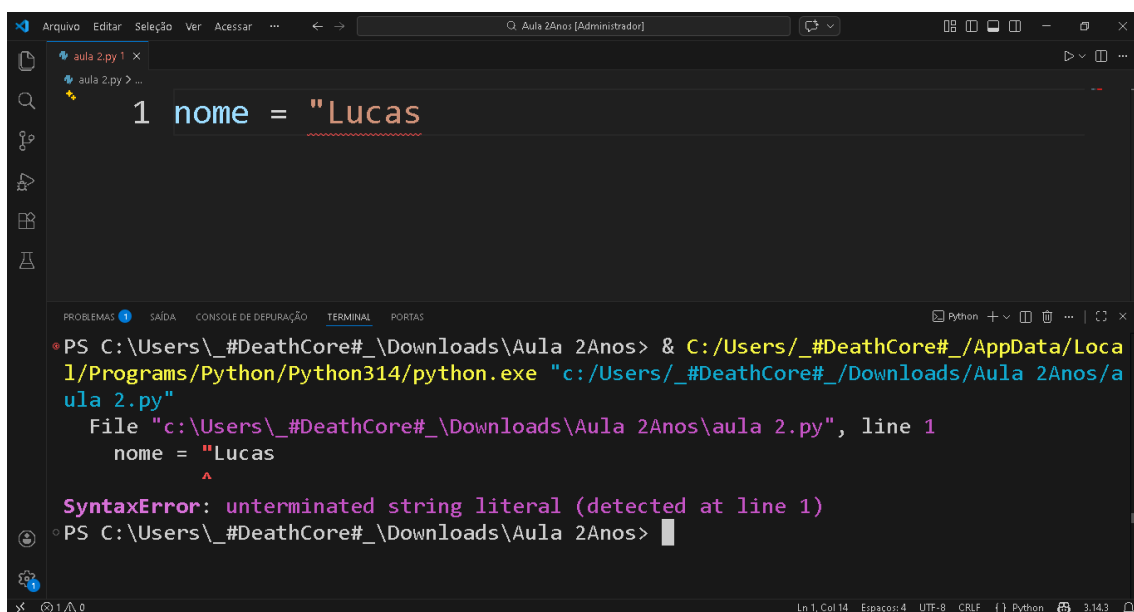
O erro de sintaxe ocorre quando a estrutura do código viola as regras formais da linguagem. Nesse caso, o interpretador identifica o problema antes mesmo de iniciar a execução. Exemplo:



The screenshot shows a Python IDE with a file named 'aula 2.py' open. The code in the editor is a single line: `1 valor =`. The terminal window at the bottom shows the command to run the file: `PS C:\Users\#DeathCore#\Downloads\Aula 2Anos> & C:/Users/#DeathCore#/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/#DeathCore#/Downloads/Aula 2Anos/aula 2.py"`. The output shows the file path and line number, followed by the error: `SyntaxError: invalid syntax`. The status bar at the bottom indicates 'Ln 1, Col 8'.

Nesse caso, a instrução está incompleta. O interpretador não consegue interpretar o que deve ser atribuído à variável.

Outro exemplo:



The screenshot shows the same Python IDE with the file 'aula 2.py'. The code in the editor is a single line: `1 nome = "Lucas`. The terminal window at the bottom shows the same command to run the file. The output shows the file path and line number, followed by the error: `SyntaxError: unterminated string literal (detected at line 1)`. The status bar at the bottom indicates 'Ln 1, Col 14'.

Aqui, as aspas não foram fechadas corretamente.

Quando ocorre um **SyntaxError (Erro de Sintaxe)**:

- O programa não inicia a execução.
- Nenhuma linha é processada.
- O problema está na estrutura formal do código.

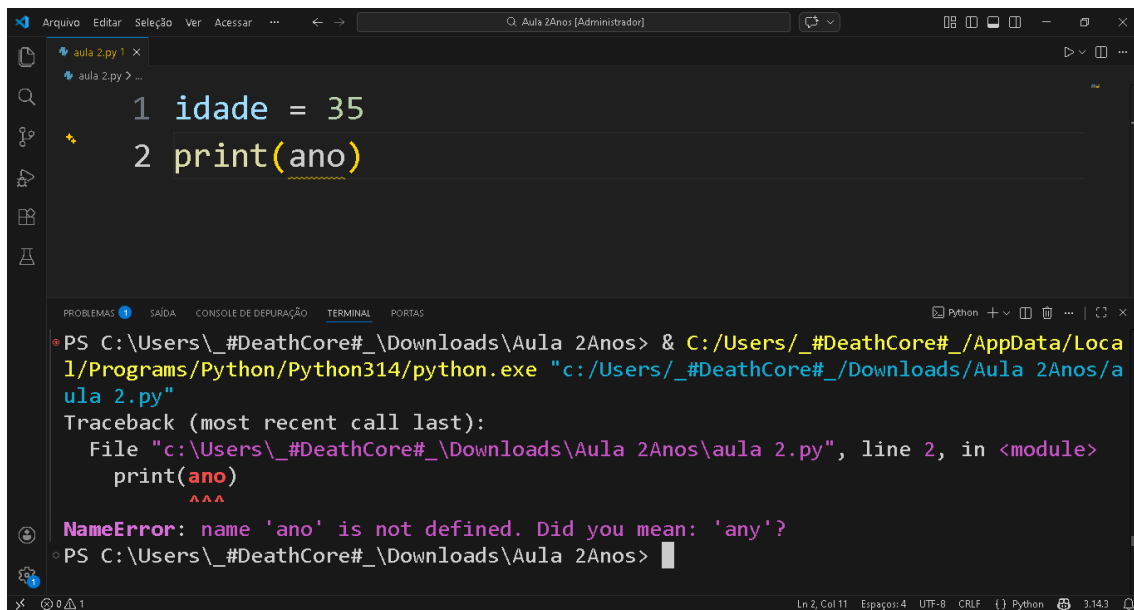
Esse tipo de erro impede completamente o funcionamento do programa até que seja corrigido.

Erro em Tempo de Execução (Runtime Error — Erro Durante a Execução)

O erro em tempo de execução ocorre quando o código está estruturalmente correto, mas uma falha acontece enquanto o programa está sendo executado.

Nesse caso, o programa inicia normalmente, executa algumas instruções e interrompe no ponto em que a falha é encontrada.

Exemplo 1 — NameError (Erro de Nome)



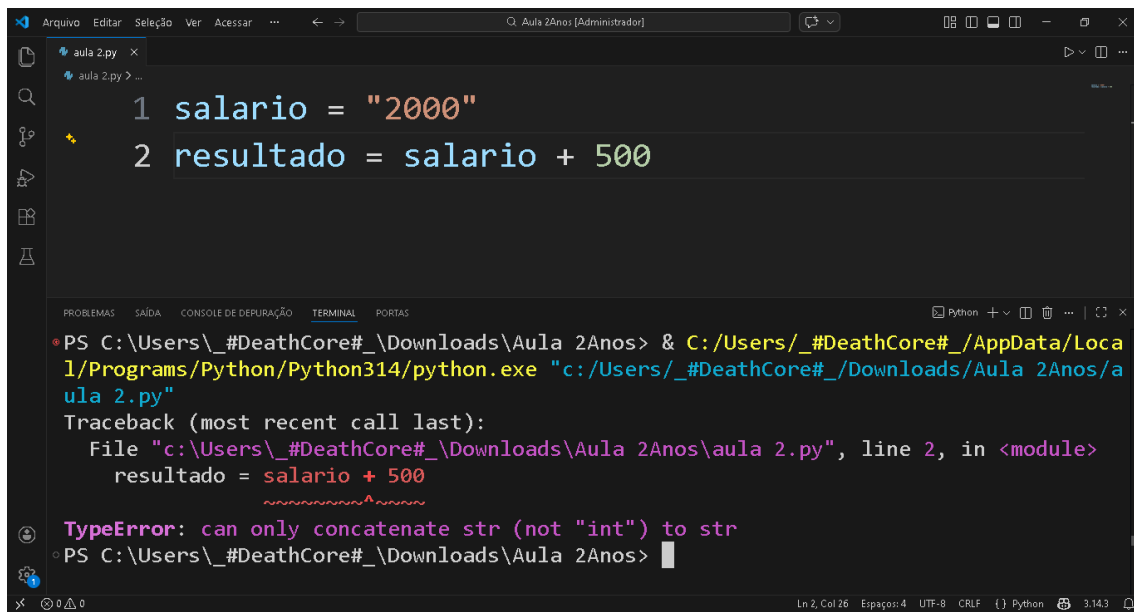
```
Arquivo  Editar  Seleção  Ver  Acessar  ...  Aula 2Anos [Administrador]

aula 2.py 1 x
aula 2.py > ...
1 idade = 35
2 print(ano)

PROBLEMAS 1 SAÍDA CONSOLE DE DEPURACÃO TERMINAL PORTAS Python + - Python 3.14.3
PS C:\Users\_#DeathCore#\Downloads\Aula 2Anos> & C:/Users/_#DeathCore#/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/_#DeathCore#/Downloads/Aula 2Anos/aula 2.py"
Traceback (most recent call last):
  File "c:\Users\_#DeathCore#\Downloads\Aula 2Anos\aula 2.py", line 2, in <module>
    print(ano)
          ^^^
NameError: name 'ano' is not defined. Did you mean: 'any'?
PS C:\Users\_#DeathCore#\Downloads\Aula 2Anos>
```

O código é estruturalmente válido. Porém, a variável ano não foi definida anteriormente. O erro ocorre apenas quando essa linha é executada.

Exemplo 2 — TypeError (Erro de Tipo)

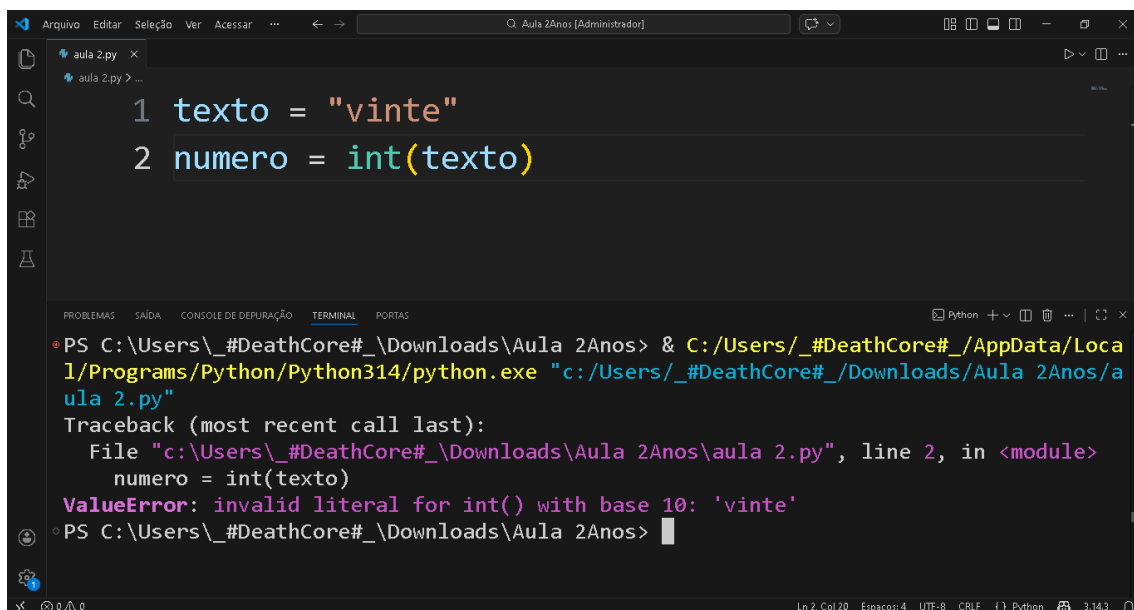


```
1 salario = "2000"
2 resultado = salario + 500
```

Traceback (most recent call last):
File "c:\Users_#DeathCore#\Downloads\Aula 2Anos\aula 2.py", line 2, in <module>
 resultado = salario + 500
TypeError: can only concatenate str (not "int") to str

O erro ocorre porque há uma tentativa de operação entre tipos incompatíveis: string e inteiro.

Exemplo 3 — ValueError (Erro de Valor)



```
1 texto = "vinte"
2 numero = int(texto)
```

Traceback (most recent call last):
File "c:\Users_#DeathCore#\Downloads\Aula 2Anos\aula 2.py", line 2, in <module>
 numero = int(texto)
ValueError: invalid literal for int() with base 10: 'vinte'

A estrutura está correta, mas a conversão falha porque o valor "vinte" não pode ser convertido para número inteiro.

Exemplo 4 — ZeroDivisionError (Erro de Divisão por Zero)

Considere o exemplo:

```
valor = 10
```

```
resultado = valor / 0
```

Ao executar esse código, o terminal exibirá algo semelhante a:

Traceback (most recent call last):

File "c:\Users\pasta\pasta\pasta\arquivo.py", line 2, in <module>

```
    resultado = valor / 0
```

```
~~~~~^~~
```

ZeroDivisionError: division by zero

Estrutura da Mensagem

A mensagem pode ser analisada em partes técnicas:

1. Traceback (most recent call last)

Indica que o interpretador está exibindo o rastreamento da execução até o ponto em que ocorreu a falha.

A expressão “most recent call last” significa que a última chamada executada é mostrada ao final do rastreamento.

2. File "c:\Users\pasta\pasta\pasta\arquivo.py", line 2, in <module>

Essa linha informa:

- O caminho completo do arquivo executado.
- O nome do arquivo (arquivo.py).
- O número da linha onde o erro ocorreu (linha 2).
- O escopo da execução (<module> indica que o erro ocorreu no escopo principal do programa).

Essa é a referência exata do ponto de falha.

3. Linha do Código que Gerou o Erro

resultado = valor / 0

O interpretador reproduz a linha exata que estava sendo executada no momento do erro.

4. Indicador Visual do Ponto Crítico

~~~~~^~~

Essa marcação é um recurso visual que indica a posição exata da expressão problemática.

No exemplo:

- O símbolo ^ aponta para a operação onde ocorreu a falha.
- O trecho sublinhado indica a expressão envolvida no erro.
- Nesse caso, a operação / 0 é o elemento que gerou a exceção.

Esse detalhe visual auxilia na identificação precisa do ponto crítico dentro da linha.

### 5. ZeroDivisionError: division by zero

A última linha apresenta duas informações fundamentais:

- **ZeroDivisionError** → tipo do erro (Erro de Divisão por Zero).
- **division by zero** → descrição específica da causa.

A leitura correta sempre começa pela última linha, pois é ali que está o diagnóstico técnico da falha.

### Exemplo com Execução Parcial

Considere o código:

```
nome = "Ana"
```

```
print(nome)
```

```
numero = int("x")
```

```
print("Fim")
```

Ao executar esse arquivo, o terminal exibirá uma mensagem semelhante a:

**Traceback (most recent call last):**

**File "c:\Users\pasta\pasta\pasta\arquivo.py", line 3, in <module>**

**numero = int("x")**

**ValueError: invalid literal for int() with base 10: 'x'**

## **Análise Técnica da Execução**

### **Etapa 1 — Execução das Primeiras Linhas**

Linha 1:

nome = "Ana"

A variável é criada corretamente.

Linha 2:

print(nome)

O valor "Ana" é exibido no terminal.

Até esse ponto, a execução ocorre normalmente.

### **Etapa 2 — Ocorrência do Erro**

Linha 3:

numero = int("x")

O interpretador tenta converter a string "x" para inteiro utilizando a função int().

Essa conversão falha porque "x" não representa um número válido.

Nesse momento ocorre o erro:

### **ValueError (Erro de Valor)**

## **Interpretação da Mensagem do Terminal**

### **1. Identificação da Linha**

File "c:\Users\pasta\pasta\pasta\arquivo.py", line 3, in <module>

Indica que o erro ocorreu na linha 3 do arquivo executado.

## 2. Linha de Código Responsável

```
numero = int("x")
```

O interpretador reproduz a instrução exata que estava sendo executada no momento da falha.

## 3. Tipo e Descrição do Erro

ValueError: invalid literal for int() with base 10: 'x'

Interpretação:

- **ValueError** → tipo do erro (Erro de Valor).
- **invalid literal for int()** → valor inválido para conversão.
- **with base 10** → tentativa de conversão para inteiro decimal.
- **'x'** → valor que causou a falha.

A mensagem informa claramente que a string 'x' não pode ser convertida para número inteiro.

## Consequência da Execução Parcial

Após a ocorrência do erro:

- A linha 4 (`print("Fim")`) não será executada.
- O programa é interrompido imediatamente na linha 3.
- O interpretador encerra o processo após exibir o erro.

## Representação do Fluxo

Linha 1 → Executa

Linha 2 → Executa

Linha 3 → Erro (ValueError)

Linha 4 → Não executa

## Procedimento Estruturado de Diagnóstico Técnico

A leitura correta da mensagem exibida no terminal não deve ser feita de forma intuitiva ou por tentativa aleatória. O diagnóstico técnico exige uma sequência lógica de análise.

Sempre que um erro surgir durante a execução, recomenda-se seguir quatro etapas formais.

## 1. Identificação do Tipo de Erro

O tipo do erro aparece na última linha do Traceback. Ele indica a categoria da falha.

Exemplos já analisados:

- `NameError` — uso de variável não definida
- `TypeError` — operação entre tipos incompatíveis
- `ValueError` — valor inválido para determinada operação
- `ZeroDivisionError` — tentativa de divisão por zero
- `SyntaxError` — erro estrutural na escrita do código

O tipo do erro direciona o raciocínio. Corrigir sem identificar o tipo da falha tende a gerar alterações imprecisas.

## 2. Localização da Linha Exata

O Traceback informa o número exato da linha onde a execução foi interrompida.

Exemplo real de terminal:

Traceback (most recent call last):

File "c:\Users\pasta\pasta\pasta\arquivo.py", line 3, in <module>

numero = int("x")

ValueError: invalid literal for int() with base 10: 'x'

Nesse caso, a linha crítica é a linha 3.

A falha não está necessariamente no início do programa nem em uma suposição subjetiva. O interpretador indica o ponto exato da interrupção.

## 3. Análise do Dado Envolvido

Após identificar o tipo e a linha, é necessário examinar:

- Qual variável está sendo utilizada?
- Qual é o tipo dessa variável?
- Qual valor está armazenado?

- Qual operação está sendo executada?

No exemplo anterior:

- Variável envolvida: valor retornado por `int("x")`
- Tipo esperado: inteiro
- Valor fornecido: string não numérica
- Operação: conversão de string para inteiro

A falha não está na estrutura do código, mas no valor fornecido à função.

#### **4. Correção da Causa**

A correção deve atuar sobre a causa da falha, não sobre seus efeitos.

Correções inadequadas incluem:

- Apagar linhas sem análise.
- Alterar variáveis aleatoriamente.
- Comentar código para “forçar” funcionamento.
- Modificar múltiplas partes simultaneamente sem diagnóstico.

Correções adequadas incluem:

- Ajustar o tipo da variável.
- Garantir que o valor seja válido para a operação.
- Definir variáveis antes de utilizá-las.
- Rever a lógica da instrução.

O objetivo não é apenas fazer o erro desaparecer, mas compreender por que ele ocorreu.

#### **O Erro como Instrumento Técnico**

A mensagem de erro não representa falha pessoal nem defeito da linguagem. Ela é um retorno técnico do interpretador indicando inconsistência entre instrução e execução.

Abordagem inadequada:

- Alterar código aleatoriamente.
- Executar repetidas vezes até “funcionar”.

- Corrigir sem compreender a causa.

Abordagem estruturada:

- Ler o tipo do erro.
- Localizar a linha exata.
- Identificar os dados envolvidos.
- Ajustar a causa da falha.

Em contextos reais, tentativa-e-erro pode gerar perda de dados, inconsistências lógicas e decisões incorretas.

O diagnóstico estruturado consolida o modelo mental do funcionamento do interpretador e fortalece a precisão técnica na escrita de código.

## **Análises Técnicas de Erros em Execução**

Os exemplos a seguir não têm como objetivo corrigir o código, mas compreender como o erro ocorre, em que ponto a execução é interrompida e qual é a natureza da falha.

Cada situação deve ser analisada segundo três critérios:

- Tipo do erro
- Linha onde ocorreu
- Motivo técnico da interrupção

### **Exemplo 1 — TypeError (Erro de Tipo)**

```
# Define quantidade como texto  
quantidade = "5"
```

```
# Soma valor numérico  
total = quantidade + 2
```

```
# Exibe total  
print(total)
```

Mensagem esperada no terminal:

```
TypeError: can only concatenate str (not "int") to str
```

### **Análise da Execução**

Linha 1 → Executa  
Linha 2 → Erro (TypeError)  
Linha 3 → Não executa

### **Interpretação Técnica**

- Tipo do erro: TypeError (Erro de Tipo)
- Linha crítica: total = quantidade + 2
- Elemento envolvido: variável quantidade
- Situação observada: tentativa de operação entre tipos incompatíveis

A execução é interrompida no momento em que a operação inválida é processada. Nenhuma instrução posterior é executada.

### **Exemplo 2 — ZeroDivisionError (Erro de Divisão por Zero)**

# Linha 1  
valor = 100

# Linha 2  
divisor = 0

# Linha 3  
resultado = valor / divisor

# Linha 4  
print(resultado)

### **Fluxo de Execução**

Linha 1 → Executa  
Linha 2 → Executa  
Linha 3 → Erro (ZeroDivisionError)  
Linha 4 → Não executa

### **Interpretação Técnica**

- Tipo do erro: ZeroDivisionError (Erro de Divisão por Zero)
- Linha crítica: resultado = valor / divisor
- Elemento envolvido: variável divisor
- Situação observada: tentativa de divisão por zero

A estrutura do código está correta. A falha ocorre durante a execução da operação matemática.

### Exemplo 3 — ValueError (Erro de Valor)

```
# Linha 1
texto = "abc"

# Linha 2
numero = int(texto)

# Linha 3
print(numero)
```

#### Fluxo de Execução

Linha 1 → Executa  
Linha 2 → Erro (ValueError)  
Linha 3 → Não executa

#### Interpretação Técnica

- Tipo do erro: ValueError (Erro de Valor)
- Linha crítica: `numero = int(texto)`
- Elemento envolvido: valor armazenado em texto
- Situação observada: tentativa de conversão inválida para inteiro

O código está estruturalmente correto. A interrupção ocorre devido ao valor fornecido à função de conversão.

### Exemplo 4 — Erro Intermediário em Execução Parcial

```
# Linha 1
preco = "200"

# Linha 2
print("Preço definido")

# Linha 3
desconto = preco * 0.1

# Linha 4
print("Desconto calculado")
```

#### Fluxo de Execução

Linha 1 → Executa  
Linha 2 → Executa

Linha 3 → Erro (TypeError)

Linha 4 → Não executa

### **Interpretação Técnica**

- Tipo do erro: TypeError (Erro de Tipo)
- Linha crítica: `desconto = preco * 0.1`
- Elemento envolvido: variável `preco`
- Situação observada: operação entre tipos incompatíveis

Nesse caso, o programa inicia normalmente e executa instruções anteriores ao erro. A falha ocorre em uma etapa intermediária, interrompendo o restante do fluxo.

### **Consolidação Técnica do Modelo de Execução e Diagnóstico**

Ao longo desta seção foi estabelecido um modelo formal do funcionamento do interpretador Python. A partir dos exemplos analisados, é possível consolidar algumas propriedades estruturais da execução:

1. A execução ocorre de forma sequencial e linear.
2. Cada linha depende do estado construído pelas instruções anteriores.
3. A interrupção ocorre exatamente no ponto da falha.
4. Nenhuma instrução posterior é executada após o erro.
5. O terminal fornece informações suficientes para localizar e classificar a falha.

Os exemplos apresentados demonstram que o erro não é um evento imprevisível. Ele segue padrões consistentes:

- Quando há problema estrutural, o programa não inicia.
- Quando há problema lógico ou de valor, o programa inicia e interrompe no ponto exato da inconsistência.
- O tipo do erro indica a natureza da falha.
- O número da linha indica o local exato da interrupção.
- A descrição final informa a causa específica.

Esse padrão reforça que o interpretador não age de forma arbitrária. Ele responde de maneira determinística às instruções escritas.

## **Erro como Parte do Processo Técnico**

A mensagem de erro não representa falha pessoal, nem indica incapacidade de programação. Ela é um mecanismo formal de comunicação do interpretador.

Ignorar a mensagem e adotar tentativa-e-erro impede a construção de um modelo mental consistente sobre:

- Tipos de dados
- Ordem de execução
- Conversões
- Operações matemáticas
- Dependência entre variáveis

O diagnóstico estruturado, por outro lado, consolida três competências fundamentais:

1. Leitura técnica de mensagens do terminal.
2. Localização precisa do ponto de falha.
3. Identificação da causa real da interrupção.

Essas competências são base para etapas futuras como:

- Validação de dados
- Controle de exceções
- Estruturas condicionais preventivas
- Programação defensiva

Antes de aprender a “tratar” erros, é necessário compreender como e por que eles ocorrem.

## **Síntese Final**

O funcionamento do interpretador pode ser resumido em três princípios centrais:

- Executa exatamente o que está escrito.
- Interrompe no ponto da falha.
- Informa tecnicamente a natureza do erro.

Compreender esse comportamento encerra a etapa fundamental da execução consciente.

A partir dessa base, torna-se possível avançar para estruturas de controle com entendimento sólido sobre:

- Fluxo de execução
- Dependência entre instruções
- Comportamento do terminal
- Leitura estruturada de falhas

O domínio desses conceitos marca a transição entre escrita intuitiva de código e análise técnica consciente da execução.

## **Exercícios — Execução Consciente e Diagnóstico Técnico**

### **Exercício 1 — Diagnóstico Completo**

Analise o código a seguir:

```
# Define valor
```

```
valor = 500
```

```
# Usa variável inexistente
```

```
print(total)
```

Responda:

- a) Qual é o tipo do erro?
- b) Em qual linha ocorre o erro?
- c) Qual é a causa da falha?
- d) Qual seria a correção adequada?

### **Exercício 2 — Classificação de Erros**

Classifique cada item indicando o tipo de erro que ocorre:

- a) numero =
- b) int("dez")
- c) print(nome)
- d) 10 / 0

### **Exercício 3 — Execução Parcial**

Analise o código:

```
# Define nome  
nome = "Ana"
```

```
# Exibe nome  
print(nome)
```

```
# Conversão inválida  
numero = int("x")
```

```
# Linha final  
print("Fim")
```

Responda:

- a) Qual linha não será executada?
- b) Qual erro ocorre?
- c) Explique o fluxo de execução até a interrupção.

#### **Exercício 4 — Representação do Fluxo de Execução**

Considere o código:

```
# Linha 1  
a = 10
```

```
# Linha 2  
b = 0
```

```
# Linha 3  
c = a / b
```

```
# Linha 4  
print(c)
```

Represente o fluxo de execução:

```
Linha 1 →  
Linha 2 →  
Linha 3 →  
Linha 4 →
```

#### **Exercício 5 — Diferença Estrutural**

Explique a diferença estrutural entre:

- `SyntaxError`
- `ZeroDivisionError`

### **Exercício 6 — Diagnóstico de Tipo**

Analise o código:

```
# Define preço como texto  
preco = "300"
```

```
# Calcula desconto  
desconto = preco * 0.1
```

Responda:

- a) Qual é o tipo do erro?
- b) Qual é a causa da falha?
- c) Qual seria a correção adequada?

### **Exercício 7 — Execução Completa**

Considere:

```
# Linha 1  
x = 5
```

```
# Linha 2  
y = 2
```

```
# Linha 3  
z = x * y
```

```
# Linha 4  
print(z)
```

Responda:

- a) O código gera erro?
- b) Justifique o fluxo de execução.

### **Exercício 8 — Análise Conceitual**

Explique por que a abordagem de tentativa-e-erro não desenvolve raciocínio técnico estruturado.

## Exercício 9 — Leitura de Traceback

Considere o código:

```
valor = "100"  
resultado = valor / 2  
print(resultado)
```

O terminal exibe:

TypeError: unsupported operand type(s) for /: 'str' and 'int'

Responda:

- a) O erro ocorre antes ou durante a execução?
- b) Qual linha é interrompida?
- c) Por que a última linha não é executada?
- d) Qual é o elemento técnico que causa a falha?

## Exercício 10 — Análise de Interrupção Sequencial

Considere:

```
a = 10  
b = 5  
print(a + b)  
  
c = int("dez")  
  
print("Processo concluído")
```

Responda:

- a) Quais linhas são executadas com sucesso?
- b) Em qual ponto ocorre a interrupção?
- c) Qual é o tipo do erro?
- d) O que esse exemplo confirma sobre o modelo sequencial do Python?

# Gabarito

## Exercício 1

- a) NameError
- b) Linha do print(total)

- c) Variável total não definida
- d) Definir total antes de utilizá-la

### **Exercício 2**

- a) SyntaxError
- b) ValueError
- c) NameError
- d) ZeroDivisionError

### **Exercício 3**

- a) print("Fim") não executa
- b) ValueError
- c) Linha 1 executa → Linha 2 executa → Linha 3 erro → Linha 4 não executa

### **Exercício 4**

- Linha 1 → OK
- Linha 2 → OK
- Linha 3 → ERRO (ZeroDivisionError)
- Linha 4 → NÃO EXECUTA

### **Exercício 5**

SyntaxError impede o início da execução; ZeroDivisionError ocorre durante a execução após o programa já ter iniciado.

### **Exercício 6**

- a) TypeError
- b) Operação entre string e float
- c) Converter preco para int ou float antes da operação

### **Exercício 7**

- a) Não gera erro
- b) Execução sequencial completa

### **Exercício 8**

Porque ignora a análise da causa da falha e impede a construção de um modelo mental estruturado do funcionamento do interpretador.

## Exercício 9

a) O erro ocorre durante a execução.

O código é estruturalmente válido, portanto o programa inicia normalmente. A falha acontece no momento em que a operação é executada.

b) A linha interrompida é:

resultado = valor / 2

c) A última linha (print(resultado)) não é executada porque o interpretador interrompe imediatamente a execução no ponto da falha. Nenhuma instrução posterior é processada após o erro.

d) O elemento técnico que causa a falha é a incompatibilidade de tipos na operação de divisão.

A variável valor é do tipo str (string) e está sendo utilizada em uma operação matemática com um int. O operador / exige operandos numéricos, e a operação entre str e int não é suportada.

## Exercício 10

a) As seguintes linhas são executadas com sucesso:

- a = 10
- b = 5
- print(a + b)

O valor 15 é exibido corretamente no terminal.

b) A interrupção ocorre na linha:

c = int("dez")

c) O tipo do erro é:

### **ValueError (Erro de Valor)**

A string "dez" não representa um número válido para conversão em inteiro.

d) Esse exemplo confirma que:

- A execução do Python é sequencial.
- O programa executa todas as linhas válidas até encontrar uma falha.
- A interrupção ocorre exatamente na linha problemática.

- Nenhuma instrução posterior (`print("Processo concluído")`) é executada após o erro.

Esse comportamento reforça o modelo formal de execução: leitura linear, interrupção imediata no ponto da falha e ausência de continuidade após o erro.